Call Graph and Module Structure

CampusSphere - Campus Opportunity Hub

Team Members:

Rajwardhan Singh (Roll No: 24CSB1A52)
Ramitha Ramesh (Roll No: 24CSB1A53)
Rayi Charan Harsha (Roll No: 24CSB1A54)

Date : 05-10-2026

Source Code :

1. Applications

```
from datetime import date
from database import save, find
from notifications import notify

def deadline_passed(opp):
    return opp["deadline"] < date.today()

def apply(student, opp_id):
    opp = find("opportunities", "id", opp_id)
    if deadline_passed(opp):
        return "Deadline over"
    application = save("applications", {"student":
student["email"],
                                        "opp_id": opp_id,
"status": "Applied"})
    notify(student, "Application submitted", "in_app")
    return application
```

2. Auth

```
from database import save, find
from helpers import is_valid_email

def register(email, password):
    if not is_valid_email(email):
        return "Invalid email"
    # real project: hash the password with bcrypt
    return save("users", {"email": email, "password":
password})

def login(email, password):
    user = find("users", "email", email)
    if user and user["password"] == password:
```

```
            return "Login OK"
    return "Login failed"
```

3. Database

```
DB = {"users": [], "opportunities": [], "applications": [],
"notifications": []}

def save(table, record):
    DB[table].append(record)
    return record

def find(table, key, value):
    for row in DB[table]:
        if row.get(key) == value:
            return row
    return None
```

4. Helpers

```
def is_valid_email(email):
    return "@" in email and "." in email

def format_date(date):
    return date.strftime("%d %b %Y")
```

5. Main

```
from datetime import date, timedelta
from auth import register, login
from applications import apply
from database import save

def handle_register(email, password):
    return register(email, password)

def handle_login(email, password):
    return login(email, password)
```

```
def handle_apply(student, opp_id):
    return apply(student, opp_id)

if __name__ == "__main__":
    save("opportunities", {"id": 1, "title": "Hackathon",
                           "deadline": date.today() +
timedelta(days=5)})
    student = handle_register("ram@college.edu", "pass123")
    print(handle_login("ram@college.edu", "pass123"))
    print(handle_apply(student, 1))
```

6. Notifications

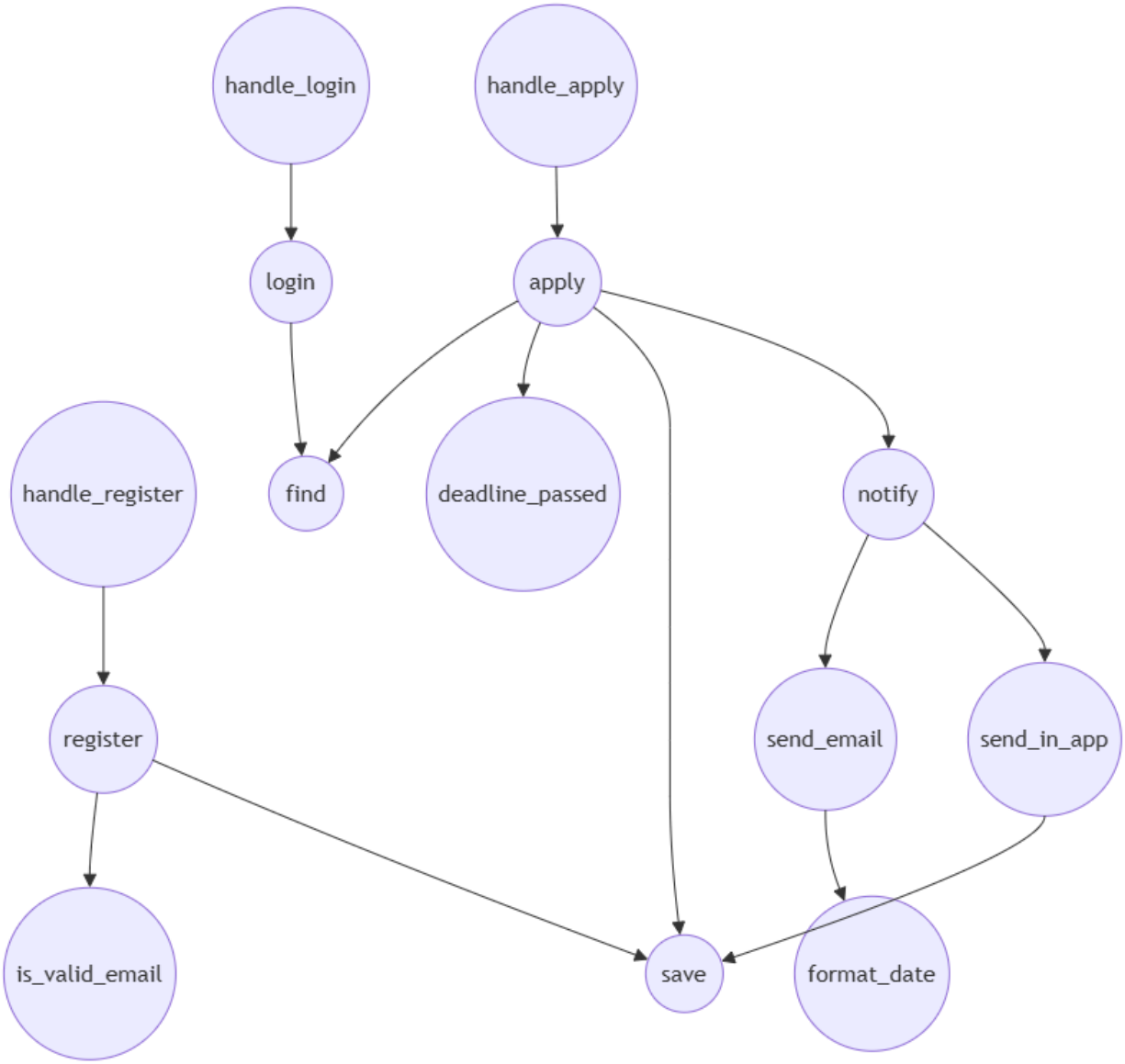
```
from datetime import date
from database import save
from helpers import format_date

def send_email(user, message):
    print("Email to", user["email"], "on",
format_date(date.today()), ":", message)

def send_in_app(user, message):
    save("notifications", {"to": user["email"], "message":
message})

def notify(user, message, channel):
    if channel == "email":
        send_email(user, message)
    else:
        send_in_app(user, message)
```

1. Call Graph Diagram



The call graph diagram, maps out the specific function-level invocations across the system's architecture. It illustrates how the entry point handlers cascade down to utility functions.

- **Entry Points (main.py):** The main module exposes handlers like handle_register, handle_login, and handle_apply, which directly invoke the core logic in their respective modules.

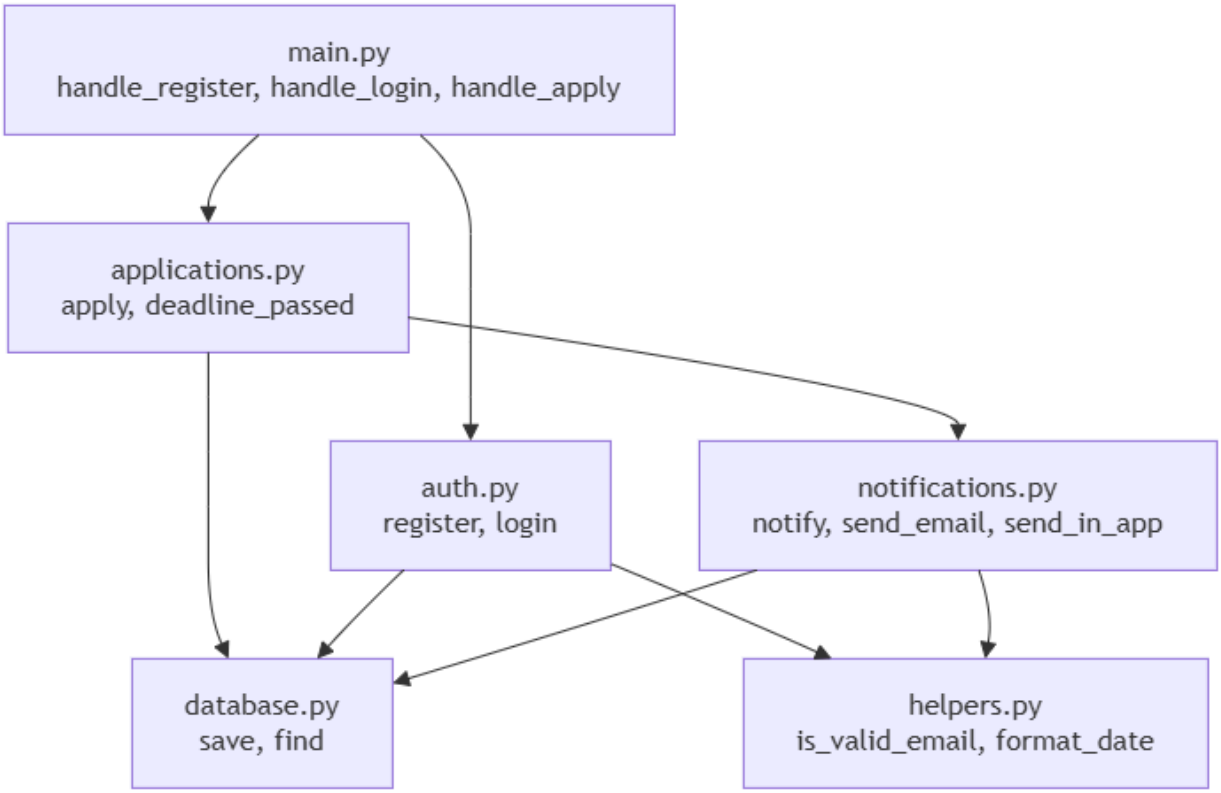
- **Authentication Flow (auth.py):** register calls is_valid_email (from helpers) and

save (from database). login interacts with find (from database).

- **Application Processing (applications.py):** The apply function acts as a central coordinator for an application submission. It fetches data via find, checks constraints via deadline_passed, commits data via save, and triggers alerts via notify.

- **Notification Routing (notifications.py):** The notify function acts as a dispatcher, branching into either send_email or send_in_app. The email function uses format_date, while the in-app notification saves the alert using save.

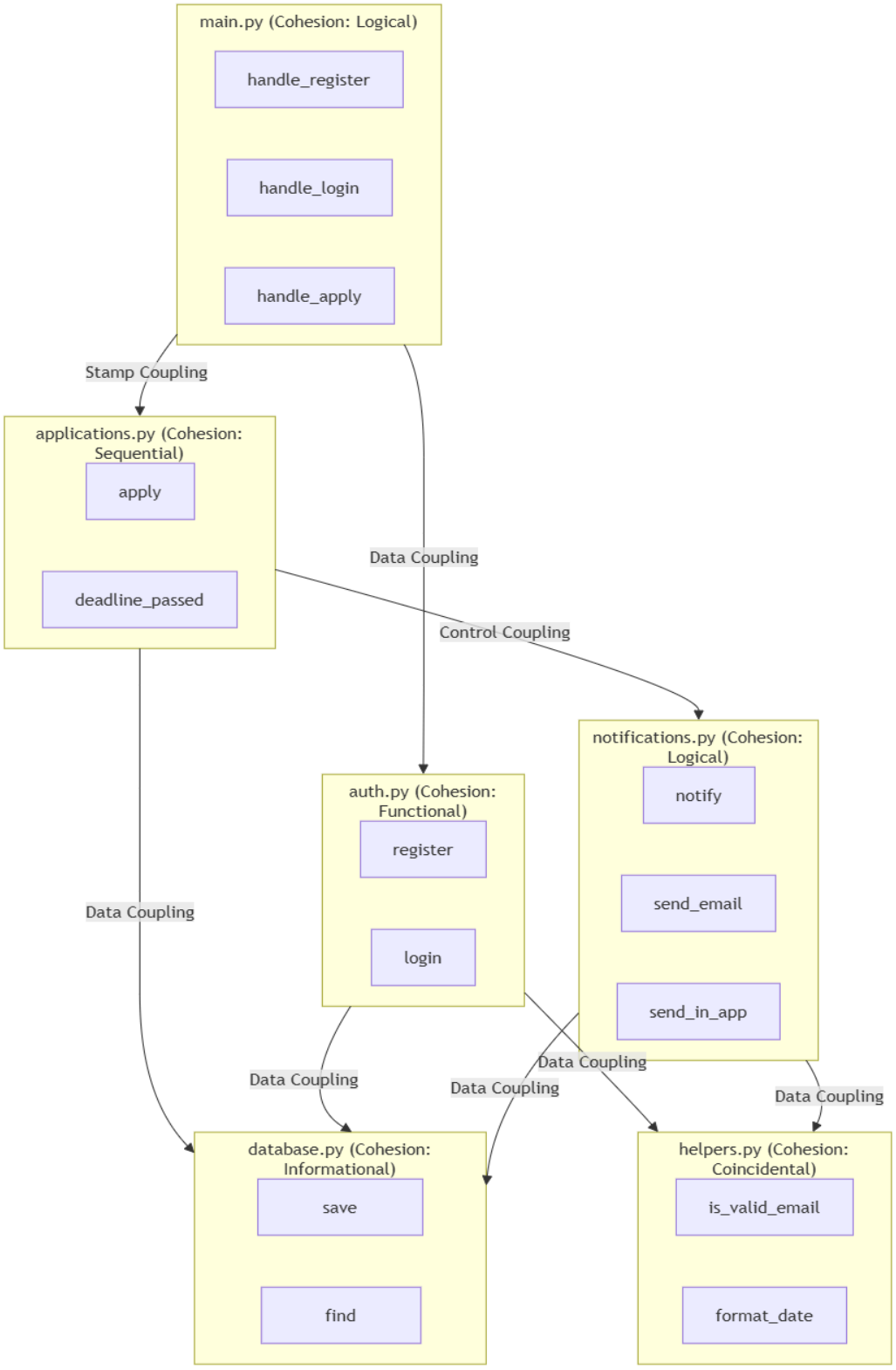
2. Module Structure



The module structure diagram, abstracts the specific function calls to show the dependencies between the physical Python files (modules).

- **High-Level Orchestration:** main.py manages the primary user workflows and depends directly on auth.py and applications.py.
- **Cross-Cutting Concerns:** The applications.py module has a broad dependency footprint, relying on notifications.py for user alerts and database.py for data persistence.
- **Shared Utilities:** database.py (handling save/find) and helpers.py (handling email validation and date formatting) act as foundational layers that multiple higher-level modules (auth, applications, notifications) depend upon.

3. Cohesion and Coupling



The diagram evaluates the structural design quality by detailing the cohesion within each module and the coupling between them.

Module Cohesion

Module	Cohesion Type	Explanation
auth.py	Functional	Highly cohesive; all functions contribute to a single, well-defined purpose (authentication).
applications.py	Sequential	Activities are linked where the output of one step (e.g., checking deadlines) feeds into the next (saving).
main.py / notifications.py	Logical	Functions are grouped by a logical category (e.g., all handlers or all notification dispatchers) rather than shared data.
database.py	Informational	Operations are grouped around a specific data structure or resource (the database).
helpers.py	Coincidental	Loosely grouped utility functions that do not share a strong meaningful relationship with one another.

Module Coupling

- **Data Coupling:** The most common and desirable form of coupling in this architecture. Seen between main.py → auth.py, auth.py → database.py/helpers.py, and notifications.py → database.py/helpers.py. Modules pass only necessary primitive data.

- **Stamp Coupling:** Exists between main.py and applications.py. This indicates that entire data structures (like an application payload) are passed, even if the receiving module only uses a portion of it.

- **Control Coupling:** Found between applications.py and notifications.py. This suggests one module passes control flags (e.g., specifying the type of notification) that dictate the internal execution path of the other.